Data Models

Document Modelling

Tarapong Sreenuch

8 February 2026

克明峻德，格物致知

Building On: Topics 2–5

Topics 2–5 produced a complete relational toolkit:

- **Topic 2:** ERD → structurally correct schema (PKs, FKs, cardinality constraints)
- **Topic 3:** BCNF normalisation eliminated redundancy; justified downshifts documented
- **Topic 4:** FWGHOS pipeline queries the schema correctly
- **Topic 5:** Constraints, transactions, indexes, and access control harden the schema

The toolkit is complete — for **transactional** workloads. But not every workload in a production system is transactional.

The Super-App: Two Workloads, One Platform

```
SUPER-APP PLATFORM

TRANSACTIONAL CORE                PERSONALISATION LAYER

Ride matching                     Passenger profiles
Payment processing                Driver profiles
Settlement                        Merchant dashboards
Promo enforcement                 Preferences & rewards

▶ ACID txns                       ▶ Aggregate reads
▶ FK integrity                    ▶ Structural variation
▶ Normalised                      ▶ Nested JSON delivery

Relational model                  ??? (this topic)
(Topics 2–5) ✓
```

The left side is solved. The right side is where the relational model starts to resist.

Failure 1: The Schema Resists Structural Variation

Three user roles — passenger, driver, merchant — each with fundamentally different profile data. The developer considers three relational options:

Option	Design	Problem
Wide table	All columns in one `UserProfile` (40+ cols)	Sparse rows; schema no longer communicates what a "driver" looks like
EAV	`UserAttribute(user_id, attr_name, attr_value)`	Type safety lost — `licenseExpiry` (date) stored as string
Subtype tables	`PassengerProfile`, `DriverProfile`, `MerchantProfile`	Rigid — new role means new table, new joins, new app logic

All three are valid relational designs. None handles what comes next.

The Combinatorial Explosion

The product team adds tier-specific fields *within* each role:

- Premium passengers: `loyaltyTier`, `loungeAccess`
- GrabCar drivers ≠ GrabFood riders: `thermalBagCertified`, `bikeType`
- Subscription merchants: `featuredPlacement`, `analyticsAccess`

Role × tier × feature flags → each combination needs nullable columns or subtype rows. Every product sprint ships a new variant; every variant ships a migration.

Result: 14 schema migrations in 6 months for profile changes alone. Two cause 30-minute read-only periods during peak hours. ~3–4 engineering-days per sprint on migration logistics.

The relational model treats structural variation as a **schema change**. The application treats it as **data**.

Failure 2: Storage Shape ≠ Delivery Shape

The mobile app needs a driver's complete profile on login — one nested JSON response:

```
Relational storage
(4 tables, 3+ JOINs)

┌─ UserProfile ──────────────┐
│ user_id │ display_name     │
│ phone   │ email            │
└────────────────────────────┘
              │ FK
┌─ DriverProfile ────────────┐
│ user_id │ vehicle_make     │
│ vehicle_model │ plate      │
│ license_number             │
│ license_expiry             │
└────────────────────────────┘
              │ FK
┌─ PreferredZones ───────────┐
│ user_id │ zone_name        │
├────────────────────────────┤
│ d-5582  │ central          │
│ d-5582  │ east             │
└────────────────────────────┘
              │ FK
┌─ TripHistory ──────────────┐
│ user_id │ trip_id          │
│ route   │ fare │ date      │
└────────────────────────────┘
```

```
Profile-load query:
SELECT ...
FROM UserProfile
INNER JOIN DriverProfile
INNER JOIN PreferredZones
INNER JOIN TripHistory
WHERE user_id = ...
→ flat rows → app re-nests JSON
  app re-nests JSON
```

```
Document storage
(1 document, 1 read)

┌─ users collection ─────────────────────────┐
│ {                                          │
│   "userId": "driver-5582",                 │
│   "displayName": "Ahmad ... ",             │
│   "phone": "+6598765432",                  │
│   "vehicle": { ... },                      │
│   "license": { ... },                      │
│   "preferences": { ... },                  │
│   "earnings": { ... },                     │
│   "subscriptionSnapshot": ...              │
│ }                                          │
└────────────────────────────────────────────┘
```

```
Profile-load query:
db.users.findOne({
 "userId": "driver-5582"
})
→ returns complete nested JSON
```

Three user types × three different JOIN chains × three re-nesting implementations — all bridging the same shape mismatch.

The Re-Nesting Tax

Each new nested structure (e.g., `incentiveProgress` inside earnings) requires updating re-nesting code for all three user types:

- **2–4 hours** of re-nesting code per user type per structural addition
- **3 user types × monthly additions → 8–12 engineering-hours/month**
- Produces **no user-visible value** — exists only to bridge the mismatch

The schema is correct. The queries are correct. The data is consistent. But the transformation layer between storage and delivery exists only because the **storage shape doesn't match the delivery shape**.

The Root Cause

The relational model is **not wrong** for this platform. The transactional core needs exactly the guarantees from Topics 2–5.

The root cause: the platform has a **second workload** — the personalisation layer — with two characteristics the relational model resists:

1. **Structural variation is routine** — new roles, tiers, features every sprint. The relational model treats this as schema change; the application treats it as data.
2. **The dominant read is an aggregate** — the app reads whole profiles as one unit. The normalised schema forces disassembly on write and reassembly on read.

This parallels Topic 3: normalisation was correct for OLTP but wrong for OLAP reporting — the answer was a second schema (star schema) shaped for the workload. Same principle, larger scale.

The Pattern

> Therefore, the engineer must **choose a storage model shaped for the personalisation workload** — not discover the mismatch after 14 migrations and months of re-nesting code.

The transactional core stays relational. The personalisation layer needs a storage shape that matches the delivery shape — different data models for different jobs.

What must this storage design guarantee?

Three Requirements for Aggregate Storage

#	Requirement	Traces to
R1	**Store data in the shape the application reads it** — one read returns the complete document; no joins, no re-nesting	Failure 2: storage-delivery shape mismatch
R2	**Accommodate structural variation without schema migration** — new roles, tiers, features are data, not schema changes	Failure 1: 14 migrations in 6 months
R3	**Make the integrity trade-offs explicit** — state what relational guarantees are preserved, lost, and how gaps are managed	Topics 2–5: hard-won guarantees must not be silently abandoned

R1 demands a new storage shape. R2 demands a new schema posture. R3 demands honesty about what is gained and what is lost.

The Document Model

Unit of storage	Document — a self-contained JSON object stored as a unit
Connection mechanism	Embedding (nesting) within documents; references (`_id`) across documents
Schema enforcement	Schema-on-read by default — "nothing rejects" on the enforcement spectrum

A **document** is a self-contained JSON object. Documents are grouped into **collections** — the document analogue of a relational table. Unlike rows, documents in the same collection can have different fields.

Every document has a unique `_id` field — the same role as Chapter 2's primary key.

JSON as the Single Notation

In the relational world, the data language and query language are separate (DDL, DML, result sets). In the document model, everything is JSON:

Role	Relational	Document Model
Define Structure	DDL (`CREATE TABLE`)	Implicit in the documents themselves
Store Data	Rows with fixed columns	JSON documents with flexible fields
Query / Filter	SQL `WHERE` clause	JSON filter: `{ "role": "driver" }`
Choose Output Fields	SQL `SELECT` list	JSON projection: `{ "displayName": 1 }`
Transform / Aggregate	SQL `GROUP BY` + functions	JSON pipeline: `[{ "$group": ... }]`

The storage shape, the query shape, and the API response shape are all the same notation.

CRUD Operations: MongoDB vs SQL

Operation	SQL (relational)	MongoDB (document)
Create	`INSERT INTO users ( ... ) VALUES ( ... )`	`db.users.insertOne({ ... })`
Read one	`SELECT * FROM users WHERE user_id = 'p-1001'`	`db.users.findOne({ "userId": "p-1001" })`
Read many	`SELECT * FROM users WHERE role = 'driver'`	`db.users.find({ "role": "driver" })`
Update	`UPDATE users SET name = 'Sarah T' WHERE ...`	`db.users.updateOne({ ... }, { "$set": { "displayName": "Sarah T" } })`
Delete	`DELETE FROM users WHERE user_id = 'p-1001'`	`db.users.deleteOne({ "userId": "p-1001" })`
Count	`SELECT COUNT(*) FROM users`	`db.users.countDocuments({})`

`$set` updates a scalar field; `$push` appends to an array. Both happen atomically within a single `updateOne`.

Aggregate and Aggregate Root

> An **aggregate** is a cluster of related objects treated as a unit for reads and writes. The **aggregate root** is the top-level entity — everything nested inside belongs to that aggregate.

The aggregate boundary defines the document boundary. In the relational model, the unit of storage is the *row* and relationships are reconstructed via joins. In the document model, the unit of storage is the *aggregate* and relationships within it are represented by nesting.

Relationships that **cross** aggregate boundaries — a user profile referencing a shared subscription plan — require references.

Embed vs Reference

Strategy	Structure	Use when	Consequence
Embed	Nest child inside parent document	Retrieved together, updated together, bounded in size	One read, one atomic write. Risk: document growth if unbounded
Reference	Store child in separate collection; parent holds child's `_id`	Shared across many parents, updated independently, or unbounded growth	Requires second query or `$lookup`. No single-document atomicity across the boundary

The decision is **workload-driven**, not structural. The same data might be embedded in one context (snapshot of plan terms at sign-up) and referenced in another (live plan definitions in the `plans` collection).

Snapshot Fields

> A **snapshot field** is an immutable copy of data captured at a point in time — recording a contractual fact, not a copy of a mutable fact.

A driver's `planNameAtSignUp` and `commissionRateAtSignUp` record what terms the driver agreed to. The app displays "You signed up under GrabCar Premium at 20% commission" — this mustn't change when the company adjusts the rate to 18%.

This is Topic 3's shipping-address snapshot in document form: label the snapshot, identify the truth, never update the snapshot. Any unlabelled duplication is a bug.

Bounded vs Unbounded Embedding

Embed when the array has a known upper bound:

- Saved places per passenger: ≤ 10
- Payment methods per user: ≤ 5
- Menu items per merchant: ≤ 100

Reference when the array could grow without limit:

- All ride history for a daily commuter over years
- All order history for a popular merchant across months

The bounded-growth rule: state the **upper bound**, the **business rule** that enforces it, and the **fallback** if exceeded. This is the document-model analogue of Chapter 2's (min, max) cardinality discipline.

Schema-on-Read vs Schema-on-Write

	Schema-on-Write (Relational)	Schema-on-Read (Document)
Structure defined	Before insert (DDL)	By the documents themselves
Variation across rows/docs	Not allowed without ALTER TABLE	Natural — each document carries its own fields
Invalid data	Rejected by the database	Accepted unless validation rules added
Schema evolution	Requires migration	New fields appear in new documents
Enforcement cost	Paid once (at write)	Paid on every read (or added via validation)

Schema-on-read makes Failure 1 disappear — adding a new role is just inserting documents with new keys. Cost: nothing prevents a misspelled `"preferedZones"` from being accepted silently.

Enforcement Spectrum — Document Model Position

```
Nothing                     Validator                   Engine
rejects                     rejects                     rejects
◀──────────────────────────────────────────────────────────────────────▶

  MongoDB                   MongoDB with               PostgreSQL
  (default)                 $jsonSchema                (DDL + constraints)
     ▲                          ▲                          ▲
     |                          |                          |
  Schema-on-read            Opt-in                     Schema-on-write
  Any JSON accepted         validation rules           Every row checked
  App interprets            reject invalid docs        Engine rejects bad data
```

MongoDB can partially close the enforcement gap with `$jsonSchema` validation — but the default posture is permissive. Enforcement must be **chosen deliberately**.

The Permissive Default

You insert `{ "displayName": "Test" }` into a MongoDB collection with **no validation rules** — no `userId`, no `role`, no type checking.

Then you insert a row into the relational `UserProfile` table without a `user_id` (defined as `NOT NULL PRIMARY KEY`).

> **Predict:** What happens in each case?

The Enforcement Spectrum in Action

> MongoDB **accepts** the document — it stores whatever JSON you provide. PostgreSQL **rejects** the insert with a NOT NULL violation.

The document stored in MongoDB is structurally valid JSON but semantically meaningless — no `userId` means no way to look up this profile. The relational system prevents this at the schema level; the document system requires either application logic or an explicit `$jsonSchema` validation rule.

```
// Adding validation — closing the enforcement gap for critical fields
db.createCollection("users_validated", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["userId", "role"],
    properties: {
      role: { enum: ["passenger", "driver", "merchant"] }
    }
  }}
})
```

Cost: the corporate admin role (`"corporate_admin"`) would be rejected until the enum is updated — validation reintroduces some migration friction.

Shape Alignment: The Driver Profile as a Document

The API response shape **is** the storage shape:

```
{
  "userId": "driver-5582",
  "displayName": "Ahmad Ibrahim",
  "phone": "+6598765432",
  "role": "driver",
  "driverType": "grabcar",
  "vehicle": {
    "make": "Toyota", "model": "Prius", "year": 2022,
    "plateNumber": "SBA1234X"
  },
  "license": { "licenseNumber": "S1234567A", "expiryDate": "2026-08-15" },
  "preferences": { "maxDistance": 25, "preferredZones": ["central", "east"] },
  "earnings": { "currentWeek": 1240.50,
    "incentiveProgress": { "ridesCompleted": 42, "target": 50, "bonus": 80.00 }
  },
  "subscriptionSnapshot": {
    "planNameAtSignUp": "GrabCar Premium",
    "commissionRateAtSignUp": 0.20, "signUpDate": "2024-01-15"
  }
}
```

`db.users.findOne({ "userId": "driver-5582" })` — one read, complete profile, no transformation.

Design A (Embedded) vs Design B (Referenced)

```
Design A — Embedded                    Design B — Referenced

┌── users ──────────────────┐            ┌── users_b ──────┐
│  userId, role             │            │  userId, role   │
│                           │            └─────────────────┘
│  ┌ vehicle: {}  ┐         │                    │ userId
│  ├ license: {}  ┤         │          ┌─────────┴─────────┐
│  ├ prefs: {}    ┤         │          ▼         ▼         ▼
│  ├ earnings: {} ┤         │     vehicles   prefs    earnings
│  └ snapshot: {} ┘         │        _b       _b        _b
└───────────────────────────┘

findOne → 1 read                       findOne + 3× $lookup → 4+ reads
Returns complete profile               Application reassembles JSON
```

Design A aligns with the profile-load workload (R1). Design B recreates the relational pattern — JOIN chains and re-assembly — inside the document model.

Three Roles, One Collection: Schema-on-Read

```
{ "userId": "passenger-1001", "role": "passenger",
  "ridePreferences": { "vehicleType": "JustGrab", "quietRide": false },
  "savedPlaces": [
    { "label": "Home", "address": "123 Clementi Ave 3" },
    { "label": "Office", "address": "1 Raffles Place" }
  ],
  "rewardPoints": 2840 }

{ "userId": "driver-5582", "role": "driver", "driverType": "grabcar",
  "vehicle": { "make": "Toyota", "model": "Prius", "plateNumber": "SBA1234X" },
  "license": { "licenseNumber": "S1234567A", "expiryDate": "2026-08-15" },
  "preferences": { "maxDistance": 25, "preferredZones": ["central", "east"] } }

{ "userId": "merchant-0891", "role": "merchant",
  "businessName": "Ah Huat Chicken Rice",
  "menu": [{ "itemName": "Roasted Chicken Rice", "price": 5.50, "category": "mains",
    "customisations": [{ "name": "Extra rice", "price": 0.50 }] }] }
```

No NULLs, no EAV, no subtype tables. Adding a fourth role is just inserting documents with new keys (R2).

Atomicity: Relational vs Document

A passenger opens the app, adds a new saved place, and earns 50 reward points simultaneously.

- In the **relational model** (Chapter 5), this update touches **two tables** (`SavedPlaces` + `UserProfile`)
- In the **document model**, both fields live inside the **same document**

> **Predict:** What construct does the relational model need to make both changes atomic? What guarantees atomicity in the document model without that construct?

Aggregate Boundary = Atomicity Boundary

> The relational model needs a **transaction** (`BEGIN`/`COMMIT` from Topic\ 5). The document model's single `updateOne` is **atomic by default** — both fields live in the same aggregate.

```
db.users.updateOne(
  { "userId": "passenger-1001" },
  { "$set": { "rewardPoints": 2890 },
    "$push": { "savedPlaces": { "label": "Gym", "address": "10 Bayfront Ave" } } }
)
// Both changes happen atomically — no BEGIN/COMMIT needed
```

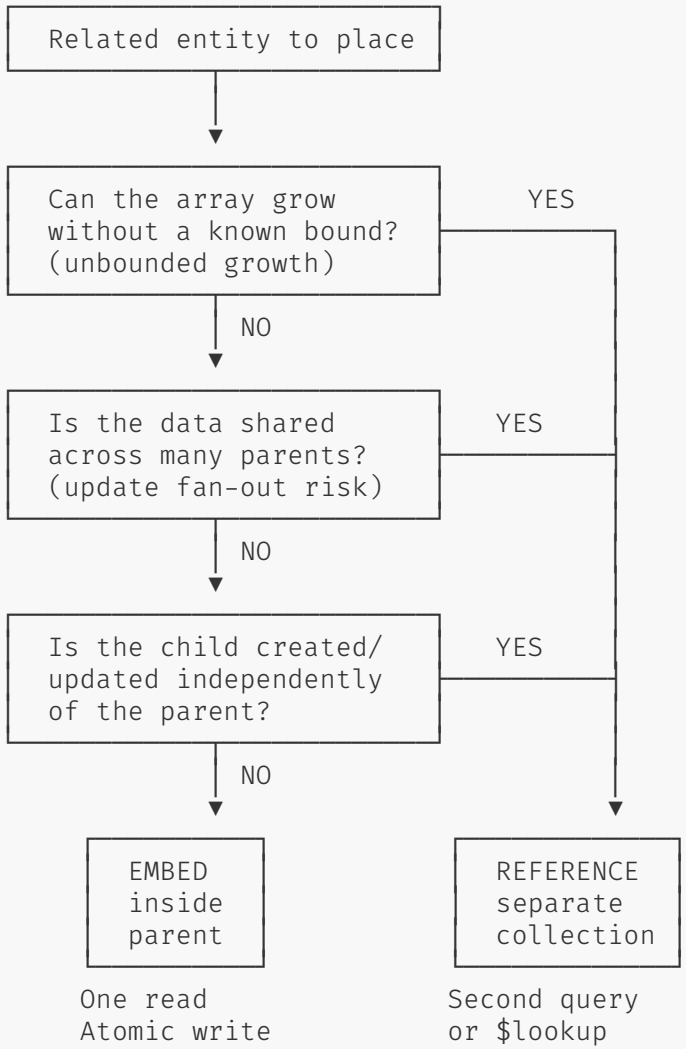
Read locality: one disk read retrieves parent + all children. **Single-document atomicity:** the aggregate boundary *is* the atomicity boundary. These are the two guarantees embedding provides.

When Embedding Breaks: Three Conditions

Condition	Test	Risk if violated
Unbounded growth	Can the array grow without a known bound?	Hits 16 MB BSON limit; read performance degrades before that
Shared data	Is the child shared across many parents?	**Update fan-out** — one logical change becomes thousands of physical writes
Independent lifecycle	Is the child created/updated independently of the parent?	Couples write paths — every child update rewrites the parent

If **any** condition is true → **reference** instead of embedding. This is the **embed-or-reference test** — a design-time decision, not a runtime check.

Embed-or-Reference Flowchart



Apply this to every related entity during document design.

Ride History: Embed or Reference?

A driver's ride history grows by 5–10 entries per day. After one year: ~2,000–3,500 entries, each with route coordinates, fare breakdown, and timestamps.

The profile-load only needs the **5 most recent trips**.

> **Predict:** Should ride history be embedded or referenced? Which condition of the embed-or-reference test applies?

Unbounded Growth → Reference

> **Referenced.** Unbounded growth — no business-rule upper bound. A 3-year driver accumulates 6,000–10,000 entries. Every profile-load would pull thousands of unnecessary entries.

Ride history belongs in a separate `rideHistory` collection, referenced by `userId`. The profile can embed `recentTrips` (last 5 — bounded) while the full history is referenced.

Costs accepted: the full-history query requires a second read or `$lookup`. The profile-load avoids the unbounded penalty.

Snapshot Discipline: Truth Map

```
TRUTH MAP — Subscription plan data

  plans collection (EDITABLE TRUTH)        users collection (IMMUTABLE SNAPSHOT)

   planId: "plan-premium"                    userId: "driver-5582"
   name: "GrabCar Premium"                   ...
   commissionRate: 0.18  ◄                   subscriptionSnapshot: {
   tier: "premium"                             planNameAtSignUp:
   monthlyPrice: 29.99                           "GrabCar Premium"
                                               commissionRateAtSignUp: 0.20
                                               signUpDate: "2024-01-15"
                                             }

              Rate changed
              from 0.20
              to 0.18

                   1 write                     0 writes  ▲
                                               Snapshot is immutable —
                                         X ——— records terms at sign-up
```

Without snapshots, changing the commission rate is 5,000 writes (one per driver). With snapshots: **1 write**. This is Chapter 3's truth-drift anomaly in document form — the fix is the same: label the copy, identify the truth, define the update rule.

Querying Nested Arrays: Two Queries, Same Data

"Find passengers where a single saved place has frequency 'daily' AND was last used before 2025-01-01."

```
// Query A — dot notation (conditions evaluated independently)
db.users.find({
  "savedPlaces.frequency": "daily",
  "savedPlaces.lastUsed": { "$lt": "2025-01-01" }
})

// Query B — $elemMatch (conditions bound to same element)
db.users.find({
  "savedPlaces": { "$elemMatch": {
    "frequency": "daily",
    "lastUsed": { "$lt": "2025-01-01" }
  }}
})
```

Sarah has two saved places: Home (frequency: "daily", lastUsed: "2025-01-20") and Office (frequency: "weekday", lastUsed: "2024-12-10").

> **Predict:** Does Query A match Sarah's document? Does Query B? Why might they differ?

`$elemMatch`: Why the Queries Differ

Query A **matches** — "daily" comes from Home, "before 2025" comes from Office. Different elements satisfy different conditions. Query B **does not match** — no single element satisfies both.

```
Element 0: { label: "Home",   frequency: "daily",   lastUsed: "2025-01-20" }
Element 1: { label: "Office", frequency: "weekday", lastUsed: "2024-12-10" }

Query A (dot notation):        Query B ($elemMatch):
"daily" ← Element 0    √      Element 0: daily √, but lastUsed ✗ (too recent)
"< 2025" ← Element 1   √      Element 1: lastUsed √, but frequency ✗ (weekday)
Both satisfied → MATCH         No single element satisfies both → NO MATCH
```

Passenger	Has "daily"?	Has lastUsed < 2025?	Same element?	Query A	Query B
Sarah Tan	Home ✓	Office ✓	No	**Match**	**No match**
Wei Ming	Home ✓	Parents ✓	No	**Match**	**No match**
Rani Devi	✗	✗	—	No match	No match

In the relational model, `WHERE` naturally binds conditions to the same row. Embedding collapses rows into arrays — `$elemMatch` recovers per-element filtering. A cost of embedding.

Regex Queries and Index Interaction

MongoDB supports regex filters on string fields. The interaction with indexes follows Topic 5's selectivity principle:

Pattern	Anchored?	Case-sensitive?	Index usage	SQL equivalent
`{ "$regex": "^\\+659" }`	✓ prefix	✓	IXSCAN — B-tree seek	`WHERE phone LIKE '+659%'`
`{ "$regex": "^Ahmad" }`	✓ prefix	✓	IXSCAN — B-tree seek	`WHERE name LIKE 'Ahmad%'`
`{ "$regex": "chicken", "$options": "i" }`	✗	✗	COLLSCAN — full scan	`WHERE LOWER(name) LIKE '%chicken%'`
`{ "$regex": "^Wei", "$options": "i" }`	✓ prefix	✗	COLLSCAN — `"i"` flag prevents B-tree	`WHERE LOWER(name) LIKE 'wei%'`

Anchored, case-sensitive regexes use the index. The `"i"` flag forces case-insensitive matching — the engine can't narrow to one case range, even with a prefix anchor.

The Aggregation Pipeline

The document equivalent of Topic 4's FWGHOS. Key differences:

- **Stage order is the author's choice** — FWGHOS has fixed logical order; the pipeline executes in the order you write it
- **`$unwind` has no SQL equivalent** — it deconstructs embedded arrays so `$group` can treat items from different documents as peers

```
[
  { "$match": { "role": "merchant" } },
  { "$unwind": "$menu" },
  { "$group": {
      "_id": "$menu.category",
      "itemCount": { "$sum": 1 },
      "avgPrice": { "$avg": "$menu.price" }
  }},
  { "$sort": { "itemCount": -1 } },
  { "$limit": 5 }
]
```

Predict: Using the seed data (2 merchants: Ah Huat with 5 menu items, Tiger Sugar with 4) — how many documents after `$match`? After `$unwind`? How many groups?

Pipeline Trace: Stage by Stage

```
Stage 1: $match { "role": "merchant" }
  7 documents → 2 documents (Ah Huat + Tiger Sugar)

Stage 2: $unwind "$menu"
  2 documents → 9 documents (one per menu item)
  Ah Huat  → 5 items (3 mains, 2 drinks)
  Tiger S  → 4 items (2 signature, 2 tea)

Stage 3: $group by "$menu.category"
  9 documents → 4 groups
  mains (3, $5.50 avg) | drinks (2, $1.35) | signature (2, $6.40) | tea (2, $6.00)

Stage 4: $sort { "itemCount": -1 }
  mains (3) → drinks (2) → signature (2) → tea (2)

Stage 5: $limit 5
  All 4 pass (fewer than limit)
```

> `$match` → 2. After `$unwind` → 9. `$group` → 4. Like Chapter 4's FWGHOS trace, but for the aggregation pipeline — and `$unwind` is the stage that has no SQL equivalent.

`$lookup`: Crossing the Aggregate Boundary

> `$lookup` is the document model's equivalent of Topic 4's LEFT JOIN — preserving all "left" documents and attaching matches from the "right" collection.

```
db.users.aggregate([
  { "$match": { "role": "merchant" } },
  { "$lookup": {
      "from": "plans",
      "localField": "subscriptionSnapshot.planNameAtSignUp",
      "foreignField": "name",
      "as": "planInfo"
  }},
  { "$unwind": "$planInfo" },
  { "$unwind": "$menu" },
  { "$group": { "_id": "$planInfo.tier",
      "avgPrice": { "$avg": "$menu.price" }, "itemCount": { "$sum": 1 } }},
  { "$sort": { "avgPrice": -1 } }
])
```

6 stages here vs 5 for the within-aggregate query. The extra `$lookup` + `$unwind` is the cost of crossing the aggregate boundary. No free lunch: embedding is the default, referencing is the escape hatch — and the escape hatch is costlier than the relational equivalent.

The Trade-Off Ledger

Guarantee	Relational (Ch 2–5)	Document Model	Status
Row/doc uniqueness	PRIMARY KEY	`_id` field	Preserved
Referential integrity	FOREIGN KEY	Not enforced — app must verify	Lost; app manages
Cross-entity constraints	CHECK, UNIQUE across tables	Within single doc only	Partially lost
Multi-step atomicity	Transactions (BEGIN/COMMIT)	Single-doc atomic; multi-doc costlier	Preserved within aggregate
No redundancy	BCNF by design	Deliberate snapshots; uncontrolled duplication is still a bug	Shifted; snapshot discipline
Ad-hoc joins	JOIN on any FK path	`$lookup` — functional but costlier	Available but costlier
Schema enforcement	DDL + constraints	Schema-on-read; opt-in `$jsonSchema`	Shifted; opt-in

For each "lost" or "shifted" guarantee, the design must name a management strategy. No strategy → incomplete design (R3).

This ledger applies to the *personalisation layer*. The transactional core retains the full relational guarantee set. A production super-app uses both: relational for transactions, documents for personalisation — **polyglot persistence**.

Callback: The Enforcement Spectrum (Now Concrete)

Earlier we asked where the document model sits on the enforcement spectrum. Now we can answer with specifics:

```
Nothing                          Validator                        Engine
rejects                          rejects                          rejects
◀──────────────────────────────────────────────────────────────────────────▶

  MongoDB                      MongoDB with                   PostgreSQL
  (default)                    $jsonSchema                    (DDL + constraints)
     ▲                              ▲                              ▲
     |                              |                              |
  { "displayName":             Missing userId                 Missing NOT NULL
    "Test" } goes in           → rejected;                    user_id → rejected
  No userId, no role,          invalid role                   before storage
  no type checking             → rejected
```

The document model trades **default enforcement** for **structural flexibility**. Each team decides how far rightward to move on the spectrum — the position is a design choice, not a given.

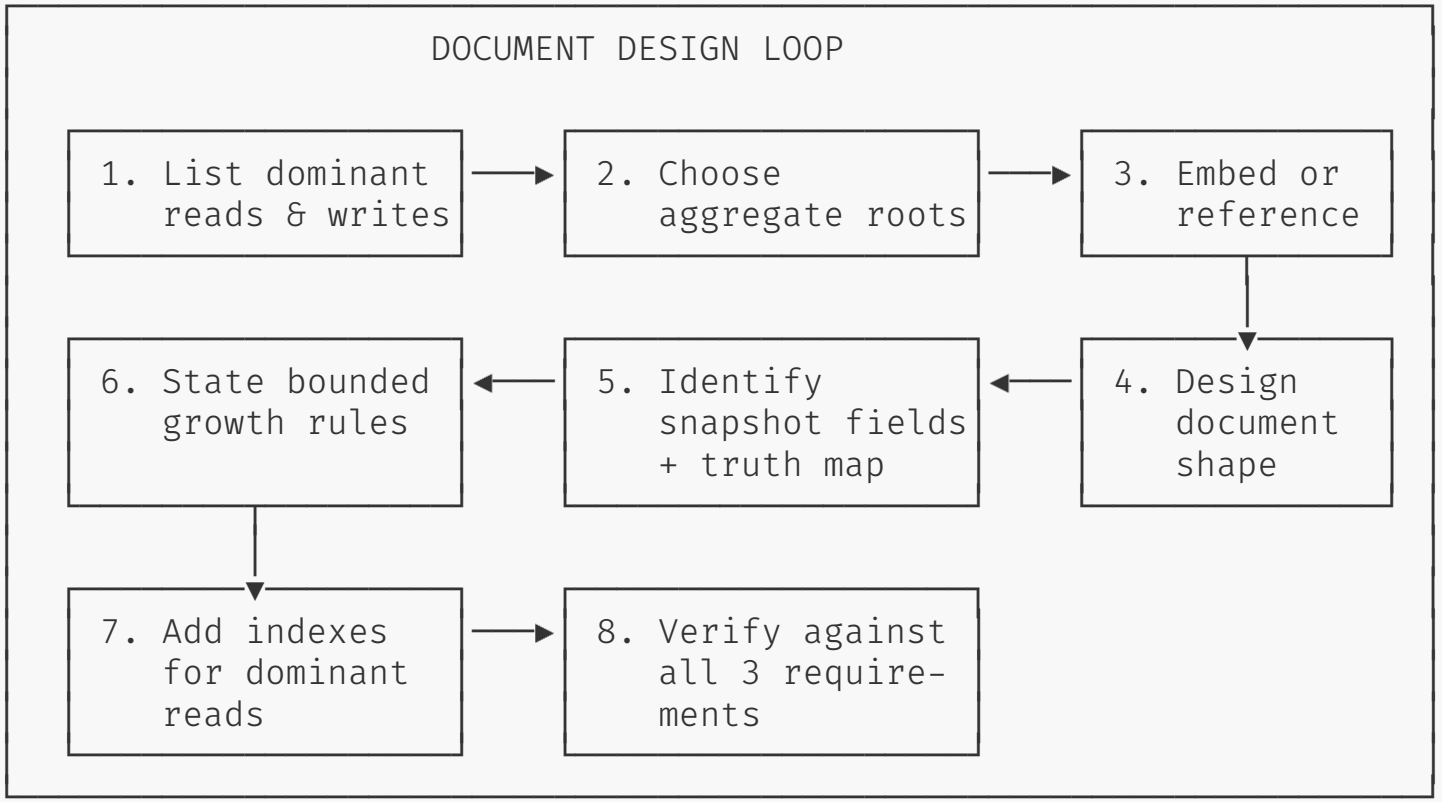
Indexes on Documents

Like Topic 5, the document model uses B-tree indexes to avoid collection scans. The same selectivity principle applies.

Index Type	MongoDB Syntax	Relational Equivalent	When To Use
Single field	`{ "phone": 1 }`	`CREATE INDEX ON users(phone)`	Exact lookups, range queries
Compound	`{ "role": 1, "license.expiryDate": 1 }`	`CREATE INDEX ON users(role, expiry)`	Multi-field filters
Unique	`{ "phone": 1 }, { unique: true }`	`UNIQUE` constraint	Enforce no-duplicate business keys
Multikey	`{ "menu.category": 1 }`	No direct equivalent	Queries on fields inside embedded arrays

Multikey indexes are new — MongoDB indexes each element of an embedded array. No relational equivalent because embedded array data would be separate rows in the relational model.

The Document Design Loop: Overview



Eight steps, each with a named outcome that feeds the next.

Steps 1–4: From Workload to Document Shape

Step	Action	Outcome
1. List dominant reads & writes	"Load driver profile on login", "Update passenger's saved places", "Add merchant menu item"	3–5 operations the design must make cheap
2. Choose aggregate root(s)	User profile = natural aggregate root; plans = separate (shared, independent)	One root per collection
3. Embed or reference each entity	Apply the embed-or-reference test to every related entity	Decision table with justification
4. Design document shape	Sketch representative JSON for each collection	The JSON the application will read

Step 4 produces the driver document (Slide 22), the role-specific documents (Slide 24), and the plans collection schema.

Embed-or-Reference Decisions: Super-App

Related entity	Decision	Justification
Vehicle details (driver)	**Embed**	Retrieved with profile, one per driver, no independent lifecycle
Licence info (driver)	**Embed**	Retrieved with profile, bounded (one), updated with driver
Saved places (passenger)	**Embed**	Retrieved every session, bounded (≤ 10)
Payment methods (all)	**Embed**	Retrieved every session, bounded (≤ 5)
Menu items (merchant)	**Embed**	Retrieved with dashboard, bounded (≤ 100)
Subscription plan (current)	**Reference**	Shared across thousands, updated independently
Subscription snapshot	**Embed**	Historical fact, immutable
Ride/order history	**Reference**	Unbounded growth

When the Document Model Is the Wrong Choice

The document model optimises for aggregate reads and writes. It's the wrong choice when:

- **The dominant workload requires joins across many independent entities** — analytics spanning orders, products, suppliers, customers. Chapter 3's star schema serves this better.
- **Referential integrity must be enforced at the database level** — financial settlement where every payment must reference a valid order. Topic 5's FK constraints are necessary.
- **The data's value lies in relationships between entities** — fraud-detection paths through transaction networks. Topics 8–10 address this.

Applying the document model to the wrong workload recreates the costs it was designed to eliminate, in reverse: instead of expensive joins to assemble data, you get expensive `$lookup` pipelines to connect it.

Steps 5–8: Discipline and Verification

Step	Action	Outcome
5. Identify snapshots + truth map	`commissionRateAtSignUp` (snapshot) ↔ `plans.commissionRate` (truth)	Truth map for every duplicated field
6. State bounded-growth rules	`savedPlaces[]` ≤ 10 (app UI enforces); `menu[]` ≤ 100 (fallback: separate collection)	Bound + rule + fallback per array
7. Add indexes	`{ "userId": 1 }`, `{ "phone": 1 }` (unique), `{ "role": 1 }`, `{ "menu.category": 1 }`	One index per dominant read
8. Verify against requirements	R1: shape aligned? R2: no migration? R3: trade-offs named?	Verified design

If any requirement fails at Step 8, revisit the step that produced the gap.

Check 1: Hospital Patient Portal (Shape Alignment)

A hospital stores patient records as documents: demographics, allergies (embedded, max ~20), current medications (embedded, max ~15), 5 most recent visits (embedded). Full visit history in a separate `visits` collection (unbounded). The portal API returns the complete patient record on login.

Can the stored document be returned directly as the API response?

> **Yes.** Demographics, allergies, medications, and recent visits are all embedded in the expected shape — the JSON-first design principle holds. Full visit history is deliberately excluded (unbounded growth → referenced) and requires a second query. The aggregate boundary matches the dominant read.

Check 2: E-Commerce Product Catalogue (Structural Variation)

An e-commerce platform stores products in a single `products` collection: books (`author`, `isbn`), electronics (`specs`, `warrantyYears`), clothing (`size`, `colour`). The business adds a new category: **digital subscriptions** (`billingCycle`, `features[]`, `trialDays`).

Does this require schema migration?

> **No.** A digital subscription is just a new document with new fields in the same collection. Schema-on-read absorbs the variation as data. Cost: if `$jsonSchema` validation uses `category: { enum: [ ... ] }`, the enum must be updated — some migration friction returns for validated fields.

Check 3: Music Streaming (Bounded Growth Under Pressure)

A music streaming service stores playlists with an embedded `tracks[]` array (bounded at 500). A user imports their personal library: **3,000 tracks** into one "All Songs" playlist.

The bound is exceeded. The fallback activates: move this playlist's tracks to a separate `playlistTracks` collection, referenced by `playlistId`.

> **Predict:** After the fallback, how many `$lookup` stages does the playlist dashboard require for a user with 12 playlists, one of which is the imported library?

Localised Fallback

> **One `$lookup` — for the imported library only.** The other 11 playlists remain embedded and load in a single read. The fallback applies only to playlists that exceed the bound, not retroactively to all playlists.

The cost of the fallback is **localised** to the playlist that triggered it. The bounded-growth rule is a design trigger, not a permanent constraint — embedded and referenced approaches coexist.

Costs accepted: one `$lookup` per over-bound playlist; increased application complexity to handle both embedded and referenced tracks.

Check 4: Tax Authority (Snapshot Discipline)

A tax authority stores filings as documents, each embedding `taxBracketAtFiling: "22%"`. The government changes the 22% bracket to 20%.

Design	Updates Required	Risk
Snapshot (immutable bracket at filing time)	1 write (update `taxBrackets` collection)	None — filed returns untouched
Current-rate (bracket references live rate)	50,001 writes (1 plan + 50,000 filings)	Miss one → incorrect tax calculation

> Snapshot discipline reduces the update from **50,001 writes to 1 write**. The filed bracket *should* reflect the rate at filing time — the same principle as `commissionRateAtSignUp`.

Check 5: Pharma Clinical Trials (Analytics Across Boundaries)

Participants embedded in `trials` (bounded ≤ 200/site). Drug suppliers in separate `suppliers` collection (shared, independent).

Within the aggregate: "Average participant age per trial site." `$match` → `$unwind` → `$group`. Three stages, one collection.

Across the boundary: "Correlation between supplier batch quality and adverse events." Requires `$lookup` to `suppliers` — added complexity and cost.

> **Predict:** Which query is cheaper in the document model? Would the cost difference exist in the relational model?

The Aggregate Boundary Is Also an Analytics Boundary

> **Within-aggregate** analytics are cheap — data lives in one collection, no `$lookup`. **Cross-aggregate** analytics pay the `$lookup` cost because data is deliberately separated.

In the relational model, both queries use JOINs at roughly uniform cost — the relational model doesn't distinguish "inside" from "outside" an aggregate. This is the flip side of fast aggregate reads: **cheap reads within the boundary, expensive reads across it**.

Check 6: E-Commerce Orders (Trade-Off Ledger)

Orders as documents with embedded line items. Each line item has `priceAtPurchase` (snapshot) and `productId` (reference).

Walk the trade-off ledger:

Guarantee	Status	Management strategy
Referential Integrity	**Lost** — `productId` might reference deleted product	App validates on write; soft-delete products
No Redundancy	**Shifted** — `priceAtPurchase` duplicates price	Snapshot is immutable; truth is `products.currentPrice`
Access Control	**Lost** — no field-level GRANT	App-layer authorisation filters fields per role

If any "lost" guarantee has no management strategy, the design fails R3. The ledger applies to *any* document design.

The Pattern Across All Six

Question type	Scenarios	Skill tested
Design decisions	1 (hospital), 2 (e-commerce catalogue)	Apply vocabulary to new domains
Design under pressure	3 (music streaming), 4 (tax authority)	Know what to do when assumptions break
Analytical limits	5 (pharma), 6 (e-commerce orders)	Honestly name what the document model cannot do cheaply

The questions are always the same: Is the shape aligned with the dominant read? Are embedded arrays bounded? Are snapshots labelled? Are lost guarantees managed? If you can answer these four for any domain, you can design a document schema for it.

Course Map

```
Topic 1 (vocabulary, decision framework, four models + validation surveyed)
│
├── Topics 2–5: Relational deep-dive
│   ├── Topic 2: ERD — capturing business rules before tables
│   ├── Topic 3: Normalisation — BCNF, star schema, justified downshifts
│   ├── Topic 4: SQL — FWHOS pipeline, JOINs, aggregation, CTEs
│   └── Topic 5: Enforcement — constraints, transactions, indexes, access control
│
├── Topics 6–10: Alternative models + cross-cutting discipline
│   ├── ▸ Topic 6: Document (MongoDB) — first alternative model;
│   │              schema-on-read, embedding vs referencing
│   ├── Topic 7: Schema validation — XSD and JSON Schema at trust boundaries
│   ├── Topic 8: RDF — knowledge graphs, identifiers, open-world assumption
│   ├── Topic 9: SPARQL — querying knowledge graphs
│   └── Topic 10: Property graph (Neo4j/Cypher) — bounded traversal
│
└── Topic 11: Capstone — multi-model design
```

Chapter 6 introduces the trade-off ledger pattern that every subsequent chapter will follow.

Concepts That Recur

Concept	Introduced Here As	Recurs In
Aggregate boundary	Document boundary = unit of read/write	Topic 10 (subgraph as traversal unit), Topic 11 (aggregate assignment)
Embed vs reference	Nesting vs cross-collection `_id`	Topic 8 (inline literal vs URI reference), Topic 10 (node property vs edge)
Snapshot discipline	Immutable copy labelled against truth source	Topic 8 (RDF literal as snapshot), Topic 11 (cross-model strategy)
Schema-on-read	Database accepts any shape; app interprets	Topic 8 (RDF open-world assumption), Topic 10 (schema-optional)
`$lookup` as LEFT JOIN	Pipeline stage preserving unmatched docs	Topic 9 (SPARQL `OPTIONAL` as LEFT JOIN equivalent)
Trade-off ledger	Preserved / lost / shifted from relational baseline	Topics 7–10 (each model's ledger), Topic 11 (combined)

Lab Preview

Topic 6 Lab — Document Modelling: Super-App Personalisation

- **6 parts**, ~120 minutes total
- **Parts 1–2:** Build two MongoDB designs — Design A (aggregate-first, embedded) and Design B (reference-heavy) — for the same super-app personalisation domain
- **Parts 3–6:** Query, compare, and analyse both designs against the same workload
- **Format:** Predict-first exercises, MongoDB queries, design reasoning

Tools: Google Colab with MongoDB (`%%mongodb` magic)

File: `06-lab-document-modelling.ipynb`